

DOCUMENT CONTROL

Document ID	EAAP-ARCH-005
Title	Functional Architecture: High-Level Design
Version	v1.0
Status	Draft
Classification	Internal Use
Parent	EAAP-CORE-001
Owner	John Goulden
Date	02.04.2026
Contact	research@johnngoulden.de

Version History

Version / Date	Author	Description
v1.0 02.04.2026	John Goulden	Initial release. ID corrected from EAAP-ARCH-003. All sections completed.

Table of Contents	
DOCUMENT CONTROL	1
Version History.....	1
Table of Contents	2
FUNCTIONAL ARCHITECTURE: HIGH-LEVEL DESIGN	3
1.1 Executive Summary	3
1.2 Purpose and Objectives.....	3
1.3 Scope	3
1.4 Audience and Roles.....	3
1.5 Requirements.....	3
1.6 Target Architecture	3
1.7 Deployment Model	4
1.8 Detailed Design.....	4
1.9 Implementation Plan	4
1.10 Test and Validation	5
1.11 Operations and Support.....	5
1.12 Security and Governance Mapping.....	5
1.13 Risks	5
Assumptions	5
Appendices	5

1.1 Executive Summary

This document defines the High-Level Design for deploying the Enterprise AI Architecture Pattern. It translates the functional architecture in EAAP-ARCH-004 into an implementable design with defined components, integration points, deployment patterns, and governance controls.

1.2 Purpose and Objectives

- Define a standard EAAP deployment architecture aligned to EAAP-ARCH-004
- Enable controlled and auditable AI adoption across enterprise domains
- Align AI capabilities with enterprise governance, ITSM, and compliance frameworks
- Support cross-domain coordination under a single governed control plane

1.3 Scope

Covers high-level design of EAAP across enterprise IT domains including infrastructure, operations, security, identity, and platform services.

Excluded: application-specific AI use cases, end-user application design. Platform-specific technology choices are in EAAP-ARCH-006.

1.4 Audience and Roles

Role	Responsibility
Architecture Owner	Defines EAAP standards. Approves design deviations.
Platform Engineering	Implements services. Manages container and Kubernetes deployments.
Domain Teams	Operate domain agents. Maintain domain knowledge bases.
Security	Enforces RBAC, audit controls, and compliance alignment.
Operations	Monitors platform health. Manages incidents and change requests.

1.5 Requirements

Functional Requirements

- Domain-specific AI capabilities across all registered domains
- Cross-domain orchestration with conflict detection and resolution
- Structured, evidence-classified outputs for all agent interactions
- Automation Gateway for controlled write actions
- ITSM integration for change, incident, and approval workflows

Non-Functional Requirements

Requirement	Target
Security	RBAC, OIDC authentication, TLS 1.3, immutable audit logs.
Auditability	All decisions and actions logged with full traceability.
Scalability	Horizontal scaling of orchestrator and domain agents independently.
Platform Independence	No dependency on specific hypervisor, cloud provider, or LLM vendor.
Air-Gap Compatibility	Full operation without internet connectivity.

1.6 Target Architecture

Layer	Components	Interface Type
Presentation	Open WebUI, REST API endpoints.	HTTPS. OIDC-authenticated.
Control Plane	EAAP Orchestrator (system prompt layer).	Internal API. Stateless.
Capability	Domain Agent containers (one per domain).	Internal API. Structured JSON.
Data	Knowledge base, vector store, telemetry, config snapshots.	Read-only from agents.
Execution	Automation Gateway (FastAPI + Celery).	Deny-by-default. Plan-approve-execute.
Governance	OpenSearch audit logs, Keycloak RBAC, Vault secrets.	Cross-layer. All interactions.

1.7 Deployment Model

Pattern	Description	Recommended For
Containerised Microservices	Stateless containers. Independent scaling.	Standard enterprise. Cloud-ready.
Kubernetes-Based	Full K8s with pod scaling and ingress.	High-concurrency. Multi-tenant. MSP.
VM-Based	Components on dedicated VMs.	Regulated. Air-gapped. Legacy infrastructure.

Full deployment specifications in EAAP-ARCH-006.

1.8 Detailed Design

Orchestrator

- Stateless control plane. No session state between invocations.
- Invoked via authenticated REST API from presentation layer.
- Produces outputs conforming to mandatory 15-field structure (EAAP-ARCH-003).

Domain Agents

- Each agent is an isolated service or container.
- Agents receive only the data required for their invocation.
- Outputs are structured JSON returned to the orchestrator.

Automation Gateway

- All write actions to infrastructure systems pass through the gateway.
- Deny-by-default with explicit allow list per action.
- Every write action requires a plan, approval token, and idempotency key.

1.9 Implementation Plan

Phase	Action	Governance Gate
Phase 1	Deploy initial domain agents. Validate outputs.	Agent validation sign-off.
Phase 2	Introduce orchestrator. Test routing.	Orchestrator routing validation.
Phase 3	Integrate Automation Gateway. Test plan-approve-execute.	Gateway integration test.
Phase 4	Apply governance controls. Connect OpenSearch audit.	Governance audit review.
Phase 5	Scale to additional domains. Validate conflict handling.	Architecture review.

1.10 Test and Validation

- **Functional validation** Each domain agent tested against a defined test case library.
- **Cross-domain orchestration testing** Multi-domain scenarios with known conflict cases.
- **Security and RBAC validation** Token scope enforcement tested across all access levels.
- **Audit log verification** All invocation records verified in OpenSearch.
- **Failure handling testing** Each failure mode from EAAP-ARCH-003A tested.

1.11 Operations and Support

- **Monitoring** Prometheus metrics. Grafana dashboards.
- **Alerting** Alertmanager with threshold-based rules.
- **Incident management** Incidents raised via ITSM. Runbooks per failure mode.
- **Change management** All platform changes follow EAAP action risk tiers and CAB process.
- **Model and agent updates** Managed via ArgoCD GitOps. Rollback maintained at all times.

1.12 Security and Governance Mapping

Framework	EAAP Control Mapping
ISO/IEC 27001:2022	A.8 Asset management via agent registry. A.9 Access control via Keycloak. A.12 Audit logging via OpenSearch.
BSI IT-Grundschutz	ORP.4 Identity management via Keycloak. OPS.1.1 Operations via ITSM integration. CON.1 Cryptography via TLS 1.3.
ITIL	Change management via plan-approve-execute. Incident management via gateway failure handling.
NIS2	Article 21 risk management via governance layer. Incident handling and access control enforced.

1.13 Risks

- **Lack of orchestration** Ungoverned, fragmented AI usage with no conflict resolution or audit trail.
- **Missing governance** Compliance, audit, and liability risk.
- **Over-automation** Reducing human oversight for Tier 2/3 actions increases operational risk.

Assumptions

- **Enterprise has ITSM and CMDB available for integration.**
- **API-based integration available for target infrastructure systems.**
- **Identity and access management infrastructure operational.**

Appendices

- **Appendix A** Architecture diagrams (component, sequence, deployment).
- **Appendix B** API specifications for orchestrator and gateway interfaces.
- **Appendix C** Governance control mappings in detail.
- **Appendix D** Domain agent test case library.
- **Appendix E** Runbook library for operational failure scenarios.